

SAUSAU Projektna dokumentacija

Predikcija cena letova

1. Kratak opis projekta

Cilj projekta je funkcionalan model za predviđanje cene avionskog leta. Ograničen je vrednostima iz skupa podataka, tačnije, može da predvidi cenu samo za informacije o letu koje su jednake sa vrednostima atributa iz skupa podataka.

Problem je rešen regresiono, gde je oprobano više modela sa različitim kombinacijama hiperparametara radi dobijanja što boljih rezultata nad dostupnim podacima. Pre toga su podaci adekvatno analizirani i pripremljeni. Čitav postupak kreiranja modela opisan je detaljno u nastavku. Model je na kraju sačuvan. Takođe, napravljen je adekvatan *API* i *User Interface*, radi lakšeg realnog primenjivanja krajnjeg proizvoda.

2. Analiza podataka

Bliže upoznavanje sa strukturom podataka i njihovom međusobnom korelacijom je bio “nulti” korak čitavog procesa.

Eksplorativna analiza je implementira kroz skriptu *data_exploration.py*, i njom su dobijene naredne informacije:

```
Oblik skupa:    300153 redova x 11 kolona
Duplikati:      0 reda
```

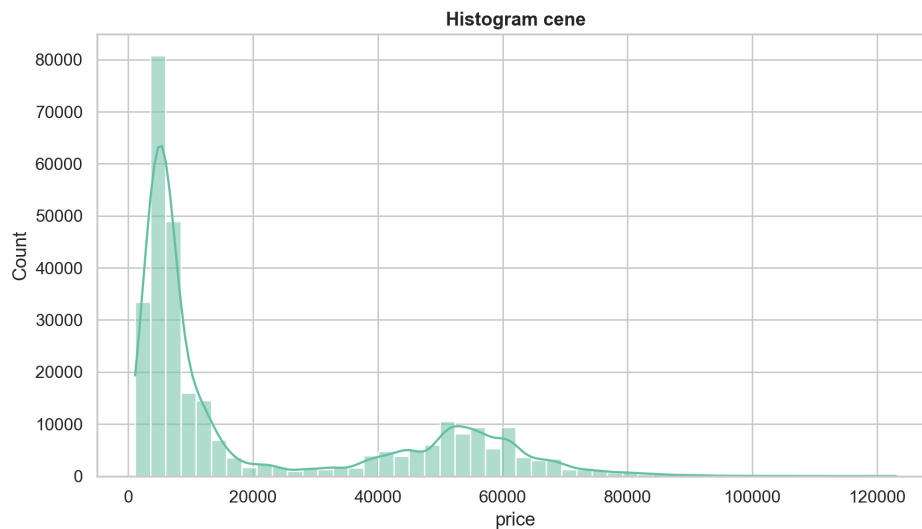
Veličina skupa podataka

Skup nema nedostajućih vrednosti. Ima 8 podataka tipa *object*, 2 tipa *int* i jedan tipa *float*.

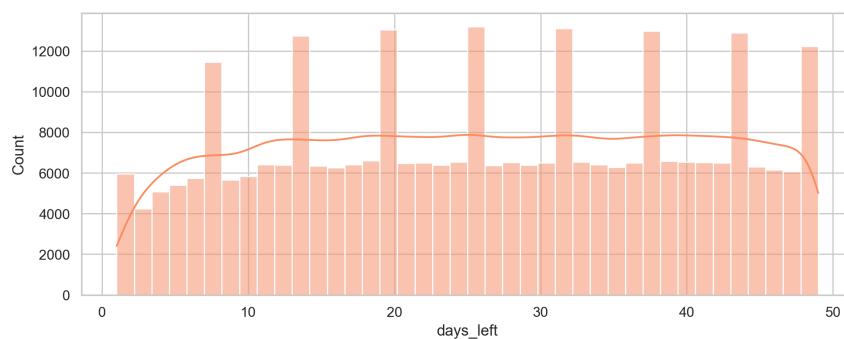
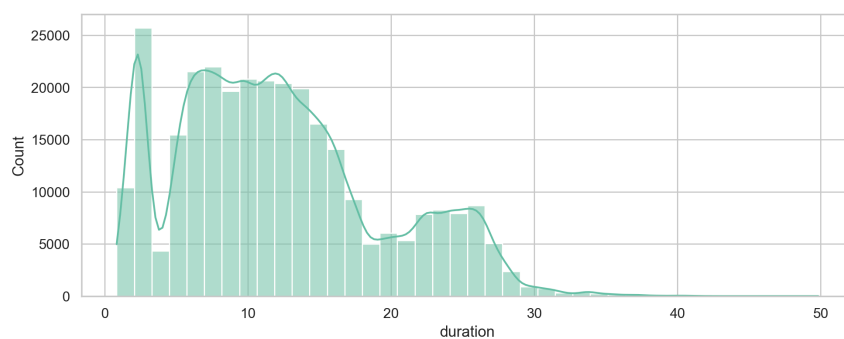
```
--- Broj jedinstvenih vrednosti po koloni ---
airline          6  → ['AirAsia', 'Air_India', 'GO_FIRST', 'Indigo', 'SpiceJet', 'Vistara']
flight          1561
source_city       6  → ['Bangalore', 'Chennai', 'Delhi', 'Hyderabad', 'Kolkata', 'Mumbai']
departure_time    6  → ['Afternoon', 'Early_Morning', 'Evening', 'Late_Night', 'Morning', 'Night']
stops            3  → ['one', 'two_or_more', 'zero']
arrival_time      6  → ['Afternoon', 'Early_Morning', 'Evening', 'Late_Night', 'Morning', 'Night']
destination_city  6  → ['Bangalore', 'Chennai', 'Delhi', 'Hyderabad', 'Kolkata', 'Mumbai']
class            2  → ['Business', 'Economy']
duration          476
days_left        49
price           12157
```

Struktura atributa

Ciljna promenljiva *price* ima minimalnu vrednost 1,105, maksimalnu 123,071, a prosečnu vrednost 20,889.66. Značajno se razlikuju vrednosti cena na osnovu klase leta. Prosečna cena za klasu *Economy* je 6572.34, a za *Business* 52540.08, dok letova *Business* klase ima broјčano znatno manje, oni uzimaju samo trećinu ukupnih podataka.



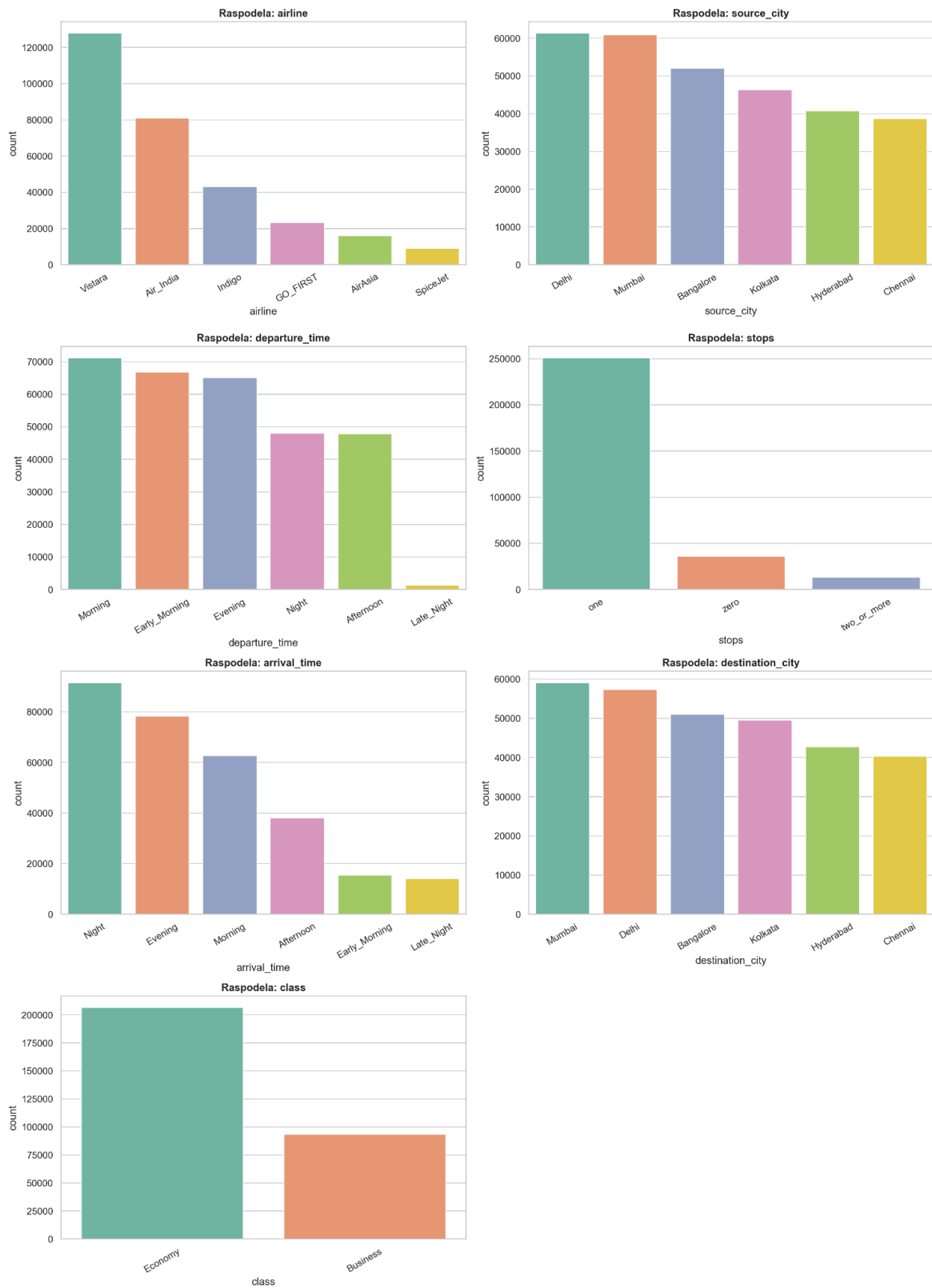
Medijana cene je 7,425



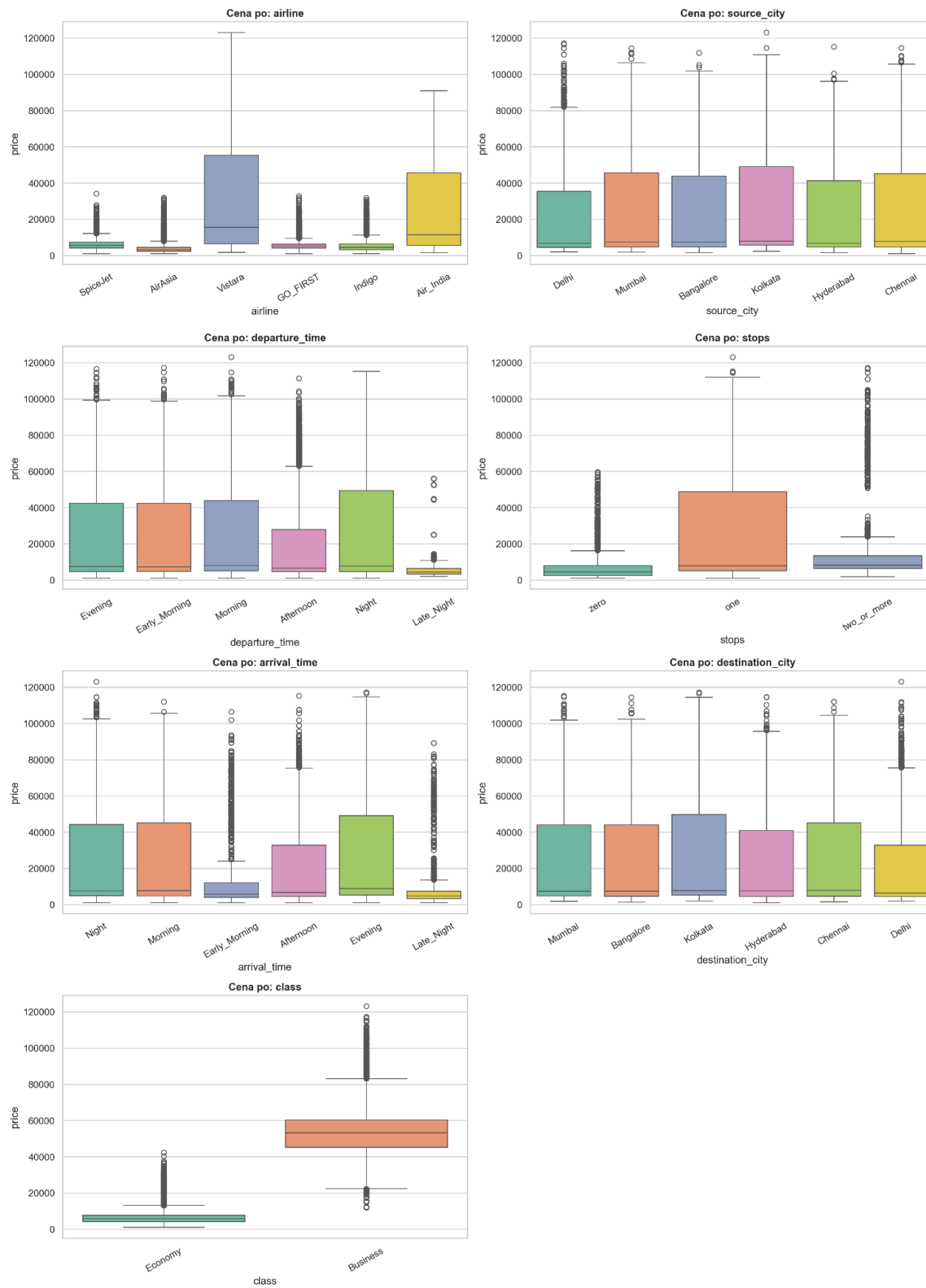
Raspodela numeričkih atributa.

Duration: Min=0.83 Max=49.83 Mean=12.22 Median=11.25

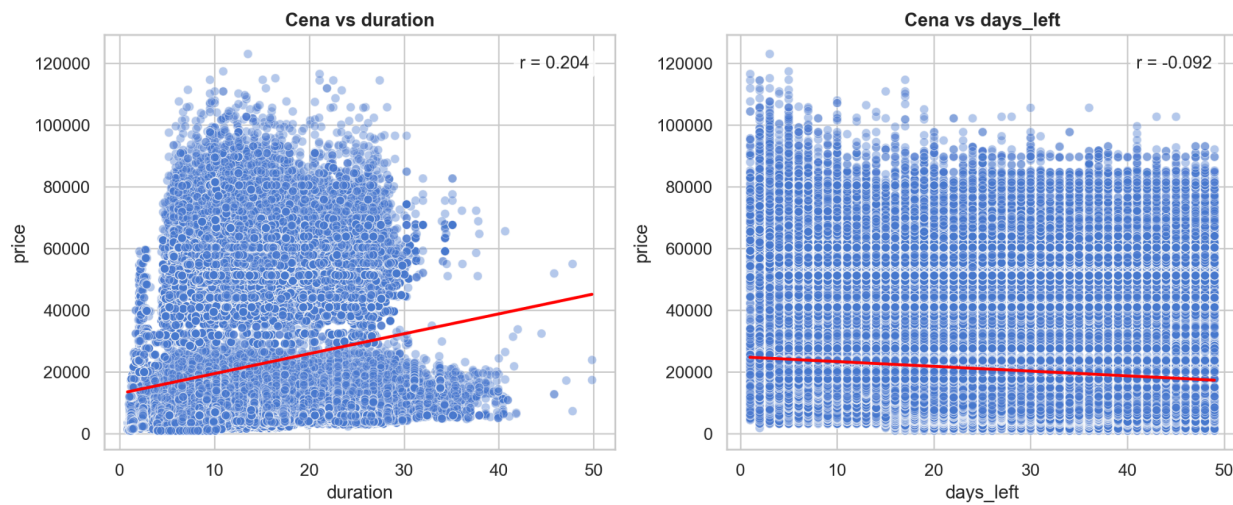
Days_left: Min=1.00 Max=49.00 Mean=26.00 Median=26.00



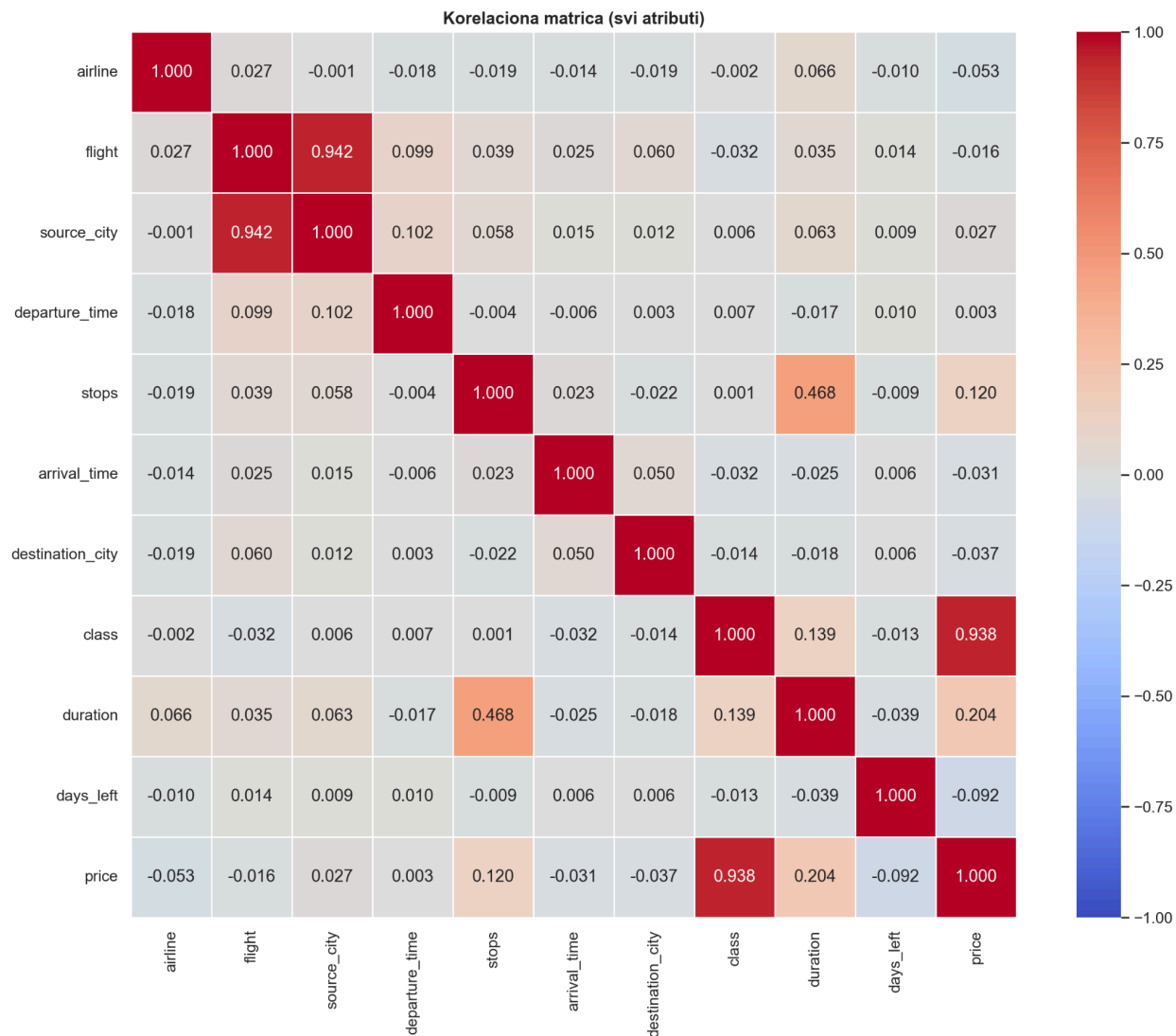
Raspodela kategorijskih varijabli kroz skup



Cena u zavisnosti od kategorijskih varijabli. Ističe se za Business klasu, letove sa jednim presedanjem i za avio kompanije Vistara i Air India



Odnos cene i numeričkih atributa. Uočava se veoma slaba korelacija (crvena linija - idealna korelacija)



Uočava se značajna korelacija između atributa duration i stop (logično), source city i flight ID (takođe logično), i duration i price. Izuzetno jako koreliraju class i price.

3. Priprema podataka

U *preprocess.py* skripti je odrađena priprema podataka. Budući da su oprobani i linearni i forest modeli, podaci su pripremljeni na dva različita načina.

Za linearne modele, odrađen je *one hot encoding* kategorijskih podataka i skaliranje numeričkih podataka. Enkoding je odrađen pre podele na train/val/test skupove, a skaliranje posle. Za linearne modele je bolji *one hot encoding* jer pretvara svaki kategorijski podatak u niz nula i jedinica, i time ne dodaje težinu nijednom podatku.

S druge strane, forest modeli bolje rade sa *ordinal* enkodovanim podacima. Tim putem se prosto svakoj vrednosti kategorijskih atributa dodeljuju brojevi 0,1,2,3... Dodatan broj kolona koji uzrokuje one hot kodovanje bi bilo nepotrebno zauzeće memorije za modele ovog tipa. Takođe, zbog prirode forest modela nije bilo potrebno skaliranje numeričkih podataka.

Odabrala sam da napravim i validacioni skup radi bolje prevencije od curenja podataka u procesu podešavanja hiperparametara. Odnos train/val/test je 70%/15%/15%.

Podela podataka je sačuvana u .npz fajl, kako bi se pri pokretanju svakog modela sprečilo ponovno pokretanje čitavog fajla za preprocesiranje. Učitavanjem skupova podataka direktno iz .npz fajla pre svakog treninga znatno skraćujemo vreme potrebno za izvršavanje programa.

4. Treniranje i modeli

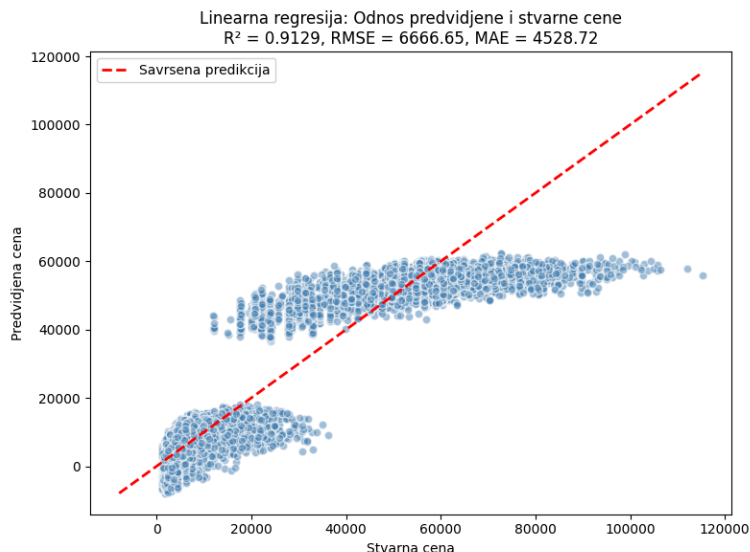
U nastavku je su opisani različiti modeli oprobani za rešavanje ovog projekta, problemi na koje sam naišla, i kako sam ih rešila.

Svaki model sam trenirala u zasebnoj skripti, u kojoj su takođe rađena podešavanja hiperparametara, iscrtavanje grafika zavisnosti predviđene i stvarne cene, računanje odabranih metrika i njihovo čuvanje u .json fajl. Za ocenu kvaliteta hiperparametra, poredile su se vrednosti RMSE za svaki model, zato što ta metrika daje lako interpretabilnu vrednost greške, i uzima u obzir jaka odstupanja, za razliku od MAE, koje jaka odstupanja “uguši” uprosečavanjem vrednosti.

4.1 Baseline - linearna regresija

Logičan početak eksperimenta je bio linearna regresija. Kako on ne zahteva podešavanje hiperparametara, validacioni skup nije korišten u ovoj skripti.

Sa MSE od 4528.72, MAE od 6666.65, i R^2 score od 0.9129, ovaj model je uzet kao baseline. Bilo koji model sa istim ili gorim metrikama se nije ni uzimao u razmatranje za odabir finalnog, najboljeg modela.

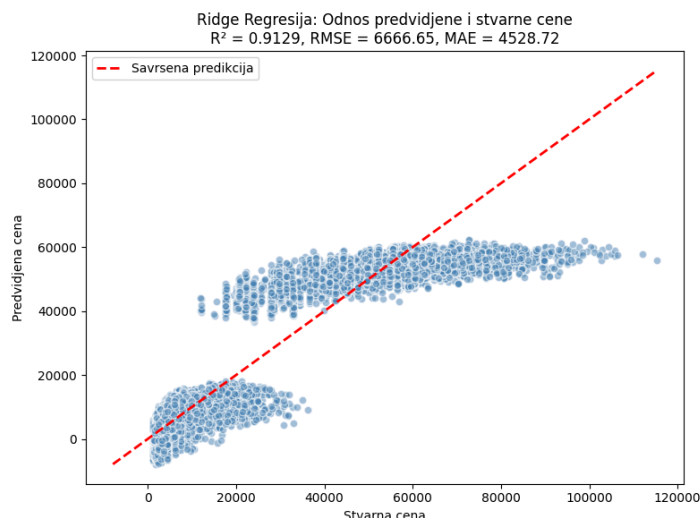


4.2 Ridge regresija

Potencijalno poboljšanje rezultata linearne regresija bi bila regularizacija podataka, to jest kažnjavanje složenih šablona predviđana. Zbog toga je implementirana i *ridge* regresija, koja uvodi L1 regularizaciju u običnu linearnu regresiju.

Optimizacija parametra alfa za stepen regularizacije je odrađena kroz for petlju. Model je validiran nad 50 ekvidistantnih vrednosti alfa u opsegu 10^{-5} do 10^5 . Najbolji rezultati se dobijaju za `alpha = 1-5`.

To je jako mala vrednost, i na osnovu toga zaključujemo da regularizacija ne unosi poboljšanje. Nijedan par atributa iz polaznog skupa nema jaku multikolineranost (ne zavise jedni od drugih). Ne postoje situacije gde se model dvoumi koji atribut da iskoristi pri podešavanju parametara. Pored toga, podataka imam dovoljno, te zbog toga ridge ne unosi poboljšanje, i veće alfa samo pogoršava predikciju. To vidimo i iz metrika ovog modela, koje su veoma bliske metrikama baseline modela.

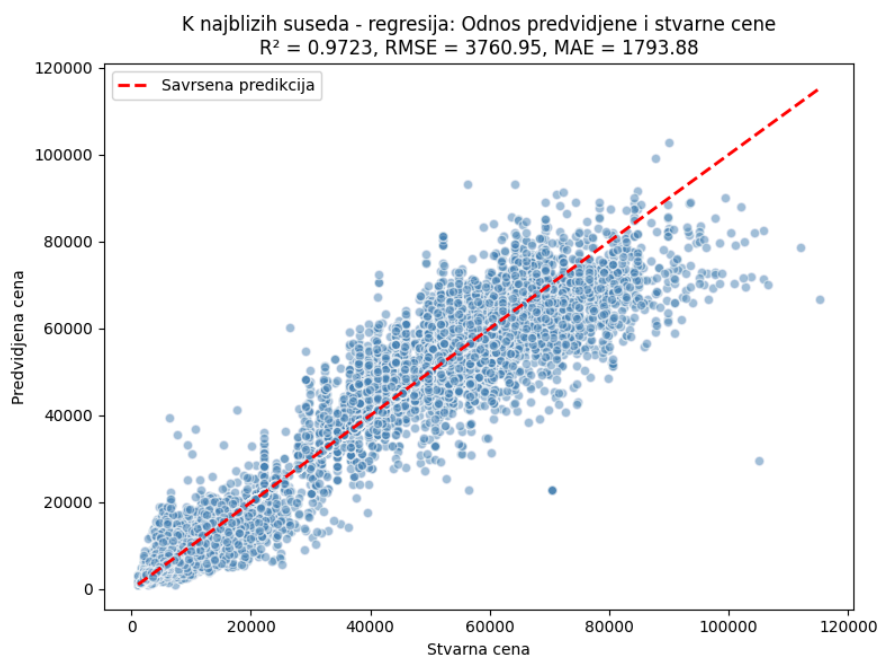


4.3 K Neighbours Regressor

Ovaj model sam htela da oprobam iz znatiželje, pošto mi je teoretski delovalo da ne bi dalo dobre rezultate za regresione probleme. Izbor broja suseda je urađen kroz for petlju, i bira se ceo broj iz intervala od 1 do 20. Najbolji rezultati su dobijeni za **K=4**. Budući da je prostim prolaskom kroz različite vrednosti K dobijena mala vrednost K kao najbolja, nisam imala potrebu da koristim lakat metodu za minimizaciju tog hiperparametra.

KNR koristi podatke pripremljene za linearne modele, kao i prethodna dva modela.

Metrike su bile značajno bolje od prethodnih, na moje iznenađenje.



U nastavku su izlistani 10 K koji daju najlošije rezultate:

```
sused = 1.000000 => RMSE = 4690.9751
sused = 19.000000 => RMSE = 4148.4777
sused = 18.000000 => RMSE = 4131.9152
sused = 17.000000 => RMSE = 4116.2194
sused = 16.000000 => RMSE = 4091.4194
sused = 15.000000 => RMSE = 4071.3455
sused = 14.000000 => RMSE = 4049.6827
sused = 13.000000 => RMSE = 4027.9868
sused = 2.000000 => RMSE = 4015.8634
sused = 12.000000 => RMSE = 4013.9537
```

4.4 Stablo odlučivanja

Budući da sam htela da podesim veći broj hiperparametara u *decision_tree.py*, pozvala sam *GridSearch* funkciju. Ona ima ugrađenu unakrsnu validaciju, za koju je ostavljena inicijalna vrednost od 5 podela.

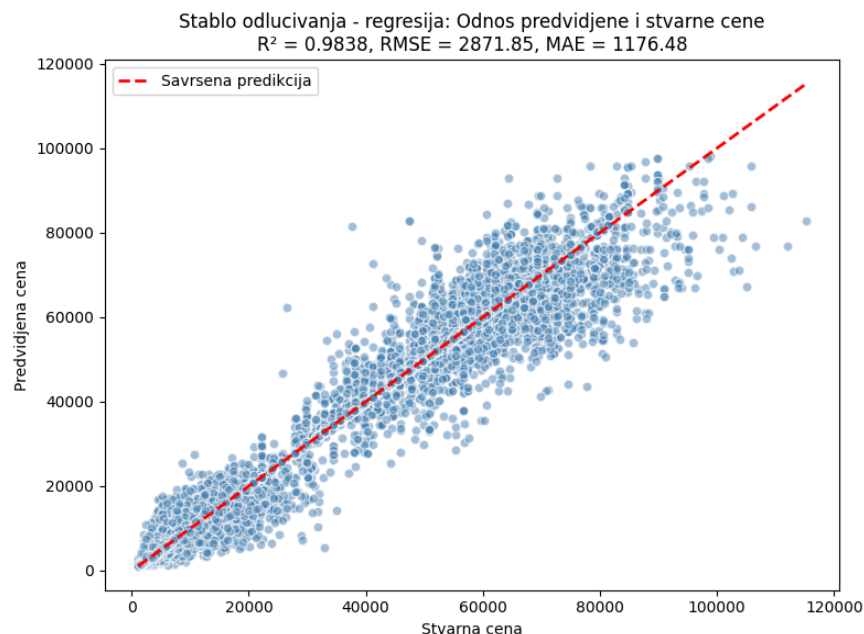
Zbog toga sam spojila trening i validacioni skup, jer nije bila potrebna moja ručna validacija. Time proširila trening skup i najverovatnije pozitivno uticala na krajnji ishod.

Hiperparametri su podešavani nad narednim intervalima:

```
params = {  
    "max_depth": [None, 3, 5, 10, 20], # maksimalna dubina stabla  
    "min_samples_split": [  
        2,  
        5,  
        10,  
        20,  
    ], # koliko uzoraka minimalno treba biti u cvoru da bi se dalje delio  
    "min_samples_leaf": [  
        1,  
        2,  
        5,  
        10,  
    ], # minimalan broj uzoraka u listu - poslednjoj iteraciji  
    "max_features": [None, "sqrt", "log2"], # max broj razmatranih  
    # featura pri podeli  
}
```

Najbolji rezultati su dobijeni sa {'max_depth': None, 'max_features': None, 'min_samples_leaf': 2, 'min_samples_split': 20}, to jest, potrebno je bilo podešavanje samo min_samples_leaf i min_samples_split na vrednosti koje nisu default.

Min_samples_split zahteva grublju podelu od podrazumevane radi boljih performansi (podrazumevana vrednost je 2).



Metrike dobijene ovim postupkom su drastično bolje nego u prethodnim slučajevima. U nastavku će se videti da su naredne metrike samo za nijansu bolje ili gore nego primenom stabla odlučivanja.

4.5 Random forest

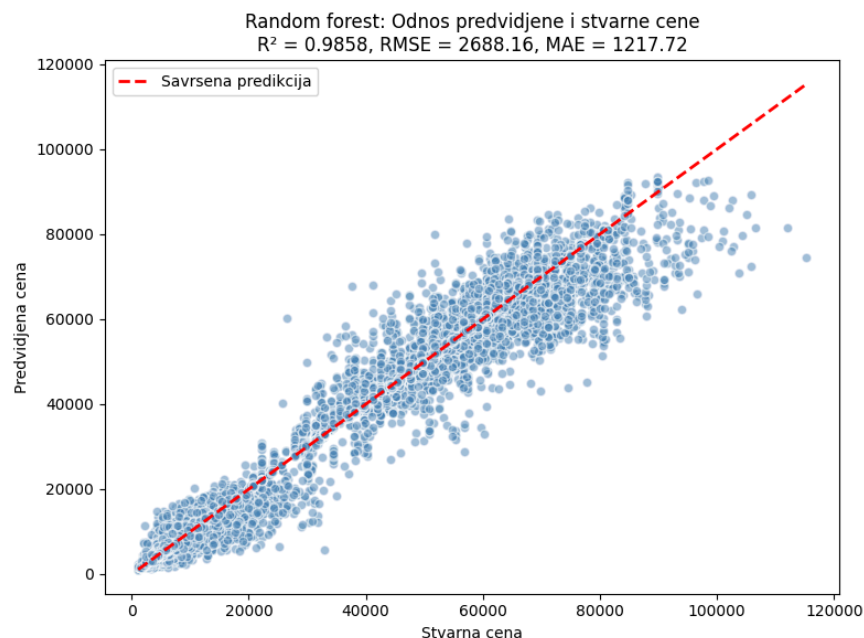
Htela sam da primenim obe ansambl metode. Za random forest su uzeti optimalni hiperparametri za stabla odlučivanja, jer se ovaj postupak svakako svodi na paralelno treniranje više stabala odlučivanja i nalaženje proseka greške. Kroz for petlju je podešavan idealan broj estimatora iz skupa [50, 100, 150, 200].

Najbolja RMSE je dobijena za `n_estimators=150`. Činjenica da optimalna vrednost nije iz ruba skupa ukazuje na to da se nije desio underfitting.

Problem na koji sam naišla - inicijalno sam pokušala da pokrenem isti GridSearch kao nad stablom odlučivanja, sa dodatim skupom za traženje optimalnog broja estimatora. To se vrtelo duže od sedam sati, i nije mi bilo jasno zašto. Razlog: Svaki fit je *Random Forest* koji gradi `n_estimators` stabala. U najgorem slučaju (`n_estimators=300`) to je 300 stabala po fitovanju. Prosečno ~ 160 stabala po fitu $\times 400$ fitova = ~ 64.000 stabala ukupno. To traje satima.

Poređenja radi, Decision Tree GridSearch je imao samo $16 \text{ kombinacija} \times 5 \text{ CV} = 80$ fitova (i to sa jednim stablom po fitu).

Zbog toga sam se odlučila za pretraživanje samo jednog hiperparametra kroz for petlju, i fiksiranje ostalih.



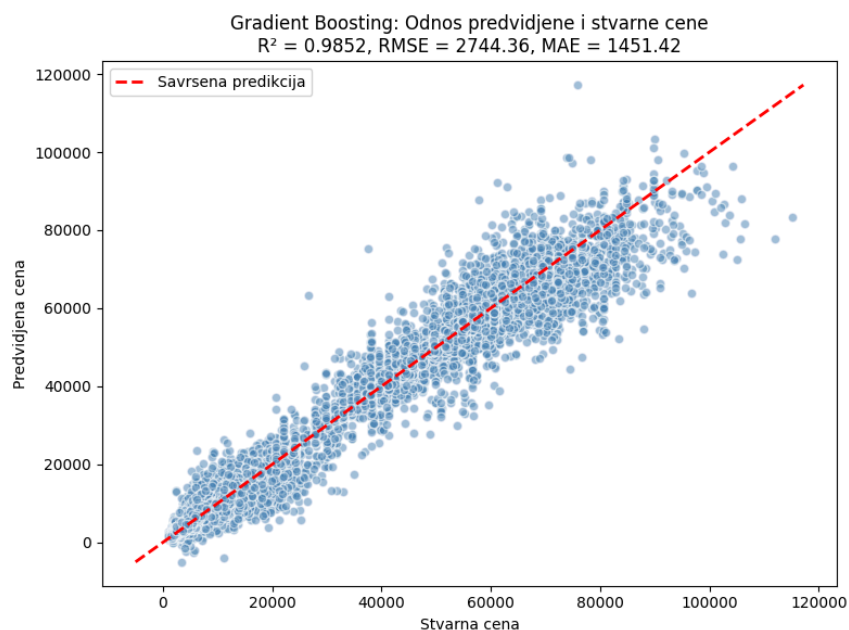
4.6 Gradient boosting

Podešavanje hiperparametara za gradient boosting je zahtevalo najviše pokušaja i probe. Poučena sedmosatnom grid pretragom nad random forest algoritmom, nisam to ni pokušavala, već sam koristila 2 for petlje za podešavanje broja estimator i koraka učenja, a ostale hiperparametre fiksirale na optimalne vrednosti dobijene stablom odlučivanja. Krajnji rezultat je time bio gori nego kod KNR-a, što nije imalo smisla.

Razlog je što gradient boosting podrazumevano gradi plitka stabla, i sa `max_samples_split=20` nijedno stablo ne stigne da nauči ništa dovoljno dobro. Vratila sam sve vrednosti na podrazumevane, i probala ponovo sa `n_estimators = [100, 150, 200, 300]` i `learning_rate = [0.05, 0.1, 0.2]`. Najbolje vrednosti su dobijene za poslednju iteraciju for petlje, što je bio pokazatelj underfittinga. Opet sam probala sa `n_estimators = [300, 400, 500]`, `learning_rate = [0.2, 0.3, 0.5]` + `max_depth = 4` (veća dubina). Opet je rubni slučaj dao najbolje vrednosti. Nije imalo smisla dalje povećavati korak učenja, te sam njega fiksirala na 0.5, i nastavila da iteriram kroz različite dubine i brojeve estimatora. Skupovi koji napokon nisu dali rubni slučaj za najbolji su `max_depth = [7, 8]` i `n_estimators = [700, 800, 900]`, a dobitna kombinacija ja `max_depth = 7` i `n_estimators = 700`.

Iako je logično da veći broj estimatora daje bolji rezultat za manji korak učenja i obrnuto, ovde su obe vrednosti velikog reda veličine dale bolji rezultat (kontra logici). Konzistentnost i veličina skupa podataka su potencijalni uzrok toga, kao i nepostojanje anomalija u skupu.

Korišten je i hiperparametar subsample, koji diktira koji postotak trening skupa vidi svako stablo. Ta količina podataka se dodeljuje stohastički. Ja sam uzela `subsample=0.8`.



5. Metrike

Metrike odabrane za ocenu ovog modela u srednja apsolutna greška, koren srednje kvadratne greške, i r^2 score. Izabrane su jer se omogućavaju najlakše tumačenje kvaliteta regresionog modela.

U nastavku je tabela metrika za sve gore opisane modele:

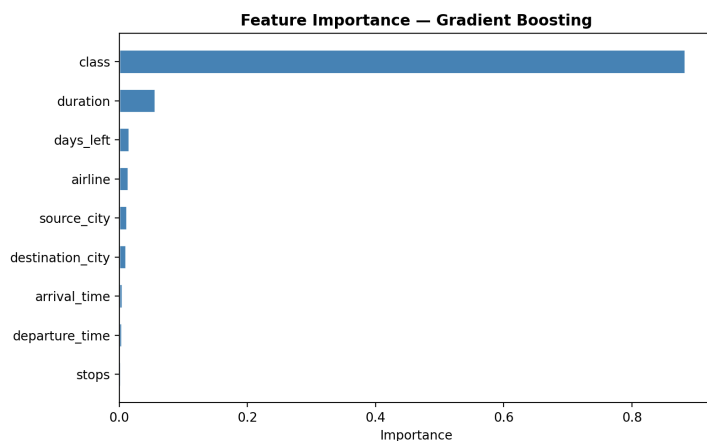
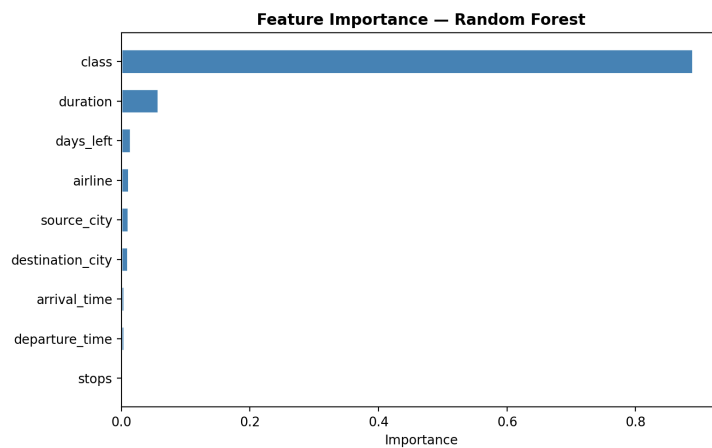
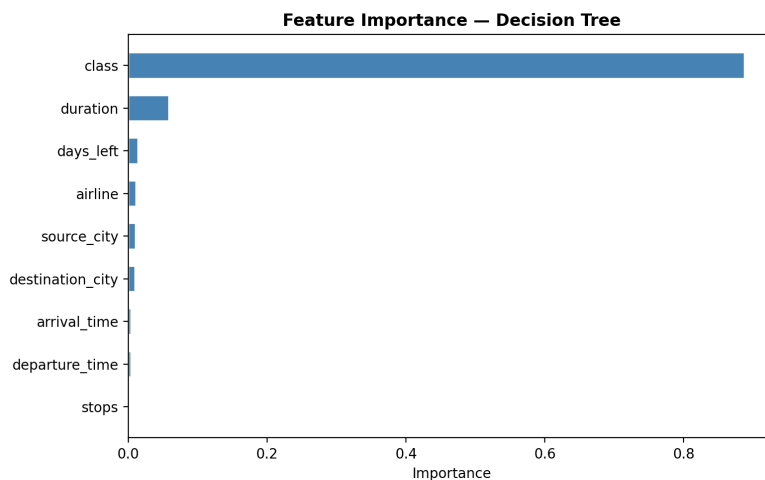
Model	MAE	RMSE	R^2
Random Forest	1216.59	2687.35	0.9858
Gradient Boosting	1451.42	2744.36	0.9852
Decision Tree	1176.38	2870.28	0.9839
K Nearest Neighbors	1793.88	3760.95	0.9723
Ridge Regression	4528.72	6666.65	0.9129
Linear Regression	4528.72	6666.65	0.9129

Svi forest modeli daju vrlo slične rezultate, bolje za nekoliko hiljada od linearnih. Zbog toga sam sva ta tri modela sačuvala radi daljeg ispitivanja. Najbolje MAE daje stablo odlučivanja, a najbolje MSE gradient boosting. Kao relevantniju metriku uzela sam RMSE, jer kvadratom kažnjava velike greške i više ih uzima u obzir nego MAE.

Da li ikako možemo dodatno poboljšati performans?

6. Najznačajniji atributi

Nalaženje najznačajnijih atributa je odrađeno nad svim forest modelima u skriptu *important_features.py*. Za sva tri su dobijeni slični rezultati:



Importance meri koliko vrednost koje klase igra ulogu u daljem deljenju čvorova stabla. Dok se kod klasifikacionih modela to kvantifikuje vrednošću Gini nečistoće, kod regresionih modela se uzima u obzir vrednost RMSE.

Definitivno najveću ulogu igra klasa kojom putnik leti, što se moglo zaključiti i iz matrice korelacije iz eksplorativne analize podataka. Takođe, taj atribut je veoma nebalansiran, jer biznis klasa čini samo trećinu svih podataka.

7. Balansiranje i najbolji model

Skrpita `balance_attempt.py` balansira atribut `class`, i tako izbalansiran dataset priprema ordinalnim enkodovanjem za forest modele. Deli podatke samo na train i test, jer su optimalni hiperparametri već pronađeni, pa nema potrebe za validacijom. Ovom skriptom samo želimo videti da li balansirani podaci daju bolji rezultat nad forest modelima nego nebalansirani.

Metrike nakon balansiranja su:

model	mae	rmse	r2 score
Gradient Boosting	1462.143	2508.925	0.98982
Random Forest	1198.097	2672.431	0.98845
Decision Tree	1130.64	2793.68	0.987378

Balansiranje je unelo poboljšanje, ne preterano veliko, ali postojeće. Zato je za implementaciju u *UI* i *API* odabran gradient boosting model treniran nad balansiranim podacima. Taj model i enkoding transformacija su sačuvani u *joblib* formatu. Pomoću *fastapi* i *streamlit* biblioteka je napravljena aplikacija spremna za korišćenje od strane drugih mašina i ljudi.

8. Zaključak i smernice

Iako su forest modeli intuitivno najsmisleniji kao rešenje klasifikacionih problema, oni su za moj regresioni problem pokazali bolje performanse nego linearni modeli. Razlog je kompleksnost skupa podataka i time teško pronalaženje linije predviđanja.

Budući da je prosečna vrednost cene iz skupa podataka blizu 21000, greška od 2500 predstavlja samo 12% te vrednosti i time skroz korektno odstupanje. Ipak, svaki korisnik bi voleo da zna ako je cena za 2500 skuplja nego što mu portal za cene letova kaže. To bi iziskivalo bolji model.

Primenjivanje grid pretrage hiperparametara nad *random forest* i *gradient boosting* algoritmima vidim kao jedan način poboljšanja performansi modela. To bi zahtevalo jaču mašinu za treniranje modela, da ne bi trening trajao nekoliko dana. Van toga, smatram da je nad ovim skupom podataka isprobana svaka tehnika za dobijanje optimalnih rezultata. Bolje predviđanje bi iziskivalo njegovo proširenje dodatnim atributima radi preciznijeg predviđanja. Predlog dodatnih atributa: cena kroz vreme (kako se cena menjala sa

približavanjem leta, uvođenje parametra datuma za svaku cenu). Takođe, da bi model postao primenljiv u stvarnom životu, morali bi se uvesti podaci za aerodrome širom sveta i raznovrsnije avio prevoznike, a UI dalje unaprediti uvođenjem zaštite od nelogičnih unosa, kao što su let od 0 sati ili let koji poleće i sleće sa istog aerodroma.